

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5115

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.
 2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.
 3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.
 4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.
 5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.
-

ANSWERS

1.

PYTHON CODE:

```
import hashlib
```

```
import time
```

```
# List of input messages
```

```
messages = [
```

```
    "Hello World",
```

```
"Cyber Security",
"Python Programming",
"Hash Functions",
"OpenAI ChatGPT"
]

print("{:<25} {:<35} {:<64}".format("Message", "MD5", "SHA-256"))
print("-" * 130)

# Measure MD5 hashing time
start_md5 = time.perf_counter()

md5_hashes = []
for msg in messages:
    md5 = hashlib.md5(msg.encode()).hexdigest()
    md5_hashes.append(md5)

end_md5 = time.perf_counter()

# Measure SHA-256 hashing time
start_sha = time.perf_counter()

sha256_hashes = []
for msg in messages:
    sha256 = hashlib.sha256(msg.encode()).hexdigest()
    sha256_hashes.append(sha256)

end_sha = time.perf_counter()
```

```

# Display results
for i in range(len(messages)):
    print("{:<25} {:<35} {:<64}".format(
        messages[i],
        md5_hashes[i],
        sha256_hashes[i]
    ))

# Performance comparison
md5_time = end_md5 - start_md5
sha_time = end_sha - start_sha

print("\nPerformance Comparison")
print("-" * 30)
print(f"MD5 Time    : {md5_time:.8f} seconds")
print(f"SHA-256 Time : {sha_time:.8f} seconds")

if md5_time < sha_time:
    print("MD5 is faster than SHA-256.")
else:
    print("SHA-256 is faster than MD5.")

```

OUTPUT:

Output

```
Message          MD5
SHA-256
-----
Hello World      b10a8db164e0754105b7a99be72e3fe5
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
Cyber Security   4690cd84196b0a4e3f84f2cc3bf79ac7
805e20cf7c3f85210a1b22b68895e2f7edff2cf1565bb685b77f87a3126db27e
Python Programming de6ae90465e75b350ed18a2024ca16ef
d15647d90cc9f8e341146110bf5058773171688b237d2f9b5704399a0b17c8bd
Hash Functions   354623b3c73aef49017cc5d8b15d85c3
c4cad6854759393c8c1fee1107af989ccb66443362dda102a199dc73dff70be5
OpenAI ChatGPT   0d07fe54b5b4d7c4f33aca9e16e358a7
0c4faf750ce9264eb8bd8020777487f581443d30c7d3279f8244b3060e340f5e

Performance Comparison
-----
MD5 Time       : 0.00002160 seconds
SHA-256 Time   : 0.00001944 seconds
SHA-256 is faster than MD5.

=== Code Execution Successful ===
```

2.

PYTHON CODE:

```
import hashlib
```

```
# RSA keys (for demonstration)
```

```
p, q = 61, 53
```

```
n = p * q
```

```
phi = (p - 1) * (q - 1)
```

```
e = 17
```

```
d = pow(e, -1, phi)
```

```
message = "Hello Deepthi"
```

```
# Create SHA-256 hash
hash_value = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n

# Sign using private key
signature = pow(hash_value, d, n)

# Verify using public key
verified_hash = pow(signature, e, n)

print("Message:", message)
print("Signature:", signature)

if hash_value == verified_hash:
    print("Signature is VALID")
    print("Integrity and Authentication confirmed.")
else:
    print("Signature is INVALID")
```

OUTPUT:

Output

```
Message: Hello Deepthi
Signature: 2029
Signature is VALID
Integrity and Authentication confirmed.
```

```
=== Code Execution Successful ===
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation